

LegalTech

Documento de Sustentacion Final

Arquitectura, stack tecnologico, pruebas, despliegue, monitoreo y demostracion funcional

Criterio de rubrica	Porcentaje	Evidencia incluida
Contexto del problema y solucion	10%	Problema, usuarios, necesidades, objetivo, alcance
Arquitectura y aspectos tecnicos	45%	Microservicios, API Gateway, SAGA, comunicacion, diagramas
Stack tecnologico	10%	Lenguajes, frameworks, BD, DevOps, cloud
Pruebas, despliegue y monitoreo	20%	Jest/Supertest, GitHub Actions, Docker, Kubernetes, Prometheus/Grafana
Demostracion funcional	15%	Ruta de demo y validacion de requerimientos minimos

Repositorio: github.com/SEBAS-TIAN34/legaltech

Frontend: <https://legaltech-ospo.vercel.app/>

Sustentacion Final - LegalTech

Este documento organiza la sustentacion segun la rubrica del curso y sirve como guion de exposicion, lista de evidencias y ruta de demostracion.

1. Contexto del problema y solucion

LegalTech atiende la necesidad de digitalizar la gestion de procesos juridicos entre clientes y abogados. En un entorno tradicional, la informacion de denuncias, casos, documentos, tiempos y facturacion suele estar dispersa en correos, archivos locales o conversaciones informales. Esto dificulta el seguimiento, la trazabilidad y la respuesta oportuna.

Usuarios involucrados

- ? Cliente: registra denuncias, consulta sus casos, revisa documentos y realiza pagos.
- ? Abogado: administra casos, clientes, documentos, tiempos e informacion de facturacion.
- ? Administrador: controla usuarios, roles y supervision general del sistema.

Necesidades detectadas

- ? Registro y autenticacion de usuarios.
- ? Separacion de experiencias entre cliente y abogado.
- ? Creacion y consulta de denuncias/casos.
- ? Gestion documental asociada a casos.
- ? Seguimiento de tiempo y facturacion.
- ? Panel de control para visualizar informacion operativa.
- ? Despliegue accesible desde la web.

Objetivo principal

Construir una plataforma web modular para centralizar la gestion legal, conectando clientes y abogados mediante servicios especializados, API Gateway, persistencia en base de datos y despliegue cloud.

Alcance

El proyecto cubre autenticacion, casos, clientes, documentos, seguimiento de tiempo, facturacion, notificaciones, dashboard, despliegue frontend en Vercel y arquitectura backend preparada para contenedores, Kubernetes, CI/CD y monitoreo.

2. Arquitectura y aspectos tecnicos

LegalTech esta disenado como una arquitectura de microservicios. Cada dominio funcional se separa en un servicio Node.js/Express con su propio modelo, rutas, controladores y Dockerfile.

Microservicios

Servicio	Puerto	Responsabilidad
auth-service	3001	Registro, login, JWT, roles y auditoria de usuarios
cases-service	3002	Creacion, consulta, actualizacion y cierre de casos
clients-service	3003	Gestion de clientes y perfiles
documents-service	3004	Carga y consulta de documentos asociados
time-tracking-service	3005	Registro de horas, tarifas y actividades
billing-service	3006	Facturas, pagos y estados de cobro
notifications-service	3007	Notificaciones del sistema
dashboard-service	3008	Agregacion de metricas funcionales
nginx gateway	80/443	API Gateway y enrutamiento hacia servicios

Comunicacion

? El frontend consume APIs REST.

? NGINX funciona como API Gateway para enrutar /api/auth, /api/cases, /api/clients, /api/documents, /api/time, /api/billing, /api/notifications y /api/dashboard.

? Los servicios comparten una base PostgreSQL/Supabase para persistencia.

? La autenticacion se basa en JWT y middlewares de autorizacion por rol.

Patron SAGA

El flujo de gestion legal puede explicarse como una SAGA coreografiada:

1. Cliente crea denuncia en cases-service.
2. Se registra auditoria del evento.
3. El caso puede generar documentos en documents-service.
4. Las horas trabajadas se registran en time-tracking-service.
5. La informacion de tiempo alimenta facturacion en billing-service.
6. notifications-service informa cambios de estado al usuario.

Compensaciones posibles:

? Si falla la creacion de documento, el caso se mantiene en estado pending y se notifica la inconsistencia.

? Si falla facturacion, el caso no se elimina; se deja pendiente de cobro.

? Si falla notificacion, el evento principal se conserva y puede reintentarse.

3. Stack tecnologico

Capa	Tecnologia	Justificacion
Frontend	HTML, CSS, JavaScript	Interfaz liviana, facil de desplegar en Vercel
Backend	Node.js, Express	APIs REST por microservicio, ecosistema amplio
Base de datos	PostgreSQL / Supabase	Modelo relacional adecuado para casos, usuarios y facturas
ORM	Sequelize	Modelado y acceso estructurado a datos
API Gateway	NGINX	Enrutamiento centralizado y desacoplamiento del frontend
Contenedores	Docker, Docker Compose	Portabilidad de servicios
Orquestacion	Kubernetes manifests	Preparacion para despliegue escalable
CI/CD	GitHub Actions	Automatizacion de build, test y despliegue
Monitoreo	Prometheus, Grafana	Metricas, dashboards y evidencia operacional
Cloud	Vercel, Render/Supabase	Despliegue web y servicios administrados

4. Pruebas, despliegue y monitoreo

Pruebas

El proyecto incluye pruebas con Jest y Supertest en servicios como auth-service, cases-service y notifications-service.

Evidencias:

- ? backend/auth-service/tests
- ? backend/cases-service/tests
- ? backend/notifications-service/tests
- ? .github/workflows/ci.yml

Comandos sugeridos:

```
cd backend/auth-service && npm test
cd backend/cases-service && npm test
cd backend/notifications-service && npm test
```

Despliegue

Evidencias:

- ? Frontend desplegado en Vercel.
- ? Repositorio conectado a GitHub.
- ? Workflows en .github/workflows.

? Dockerfiles por microservicio.

? docker-compose.nginx.yml para entorno integrado.

? Manifiestos Kubernetes en k8s.

Flujo CI/CD:

1. Cambio local.
2. git add, git commit, git push origin main.
3. GitHub Actions ejecuta validaciones/build.
4. Vercel despliega frontend automaticamente.

Monitoreo

Se agrega una base de monitoreo en:

```
? docker-compose.monitoring.yml
? monitoring/prometheus/prometheus.yml
? monitoring/grafana/provisioning/datasources/prometheus.yml
? monitoring/grafana/dashboards/legaltech-overview.json
```

Comando:

```
docker compose -f docker-compose.monitoring.yml up -d
```

Servicios:

```
? Prometheus: http://localhost:9090
? Grafana: http://localhost:3000 usuario admin, clave admin
```

5. Demostracion funcional

Ruta recomendada para la demo:

1. Registro de usuario cliente con correo nuevo.
2. Inicio de sesion.
3. Creacion de denuncia/caso desde el panel cliente.
4. Consulta del caso creado en "Mis Casos".
5. Ingreso como abogado para visualizar casos.
6. Creacion/consulta de documentos o facturas.
7. Mostrar manejo de error: correo duplicado o politica RLS.
8. Mostrar repositorio, workflow CI/CD y evidencia de despliegue.
9. Mostrar Prometheus/Grafana si esta levantado localmente.

Requerimientos minimos de la rubrica

Requerimiento	Como demostrarlo
Creacion de informacion	Registrar usuario, crear denuncia/caso
Consulta de informacion	Ver casos, clientes, documentos o facturas
Actualizacion de informacion	Cambiar estado de caso/factura
Eliminacion o cancelacion	Cerrar/cancelar caso o anular factura
Comunicacion entre microservicios	Explicar flujo gateway-servicios y dashboard agregador

Preguntas probables y respuestas cortas

Por que microservicios?

Porque cada dominio legal tiene responsabilidades separadas: autenticacion, casos, documentos, facturacion y notificaciones pueden evolucionar, probarse y desplegarse de forma independiente.

Cual es el rol del API Gateway?

Centraliza la entrada al backend y evita que el frontend conozca la ubicacion interna de cada servicio.

Como se maneja autenticacion?

El servicio de autenticacion genera tokens JWT y los middlewares validan identidad y roles en los servicios protegidos.

Donde se aplica SAGA?

En los procesos que cruzan varios dominios, como denuncia -> documentos -> tiempos -> facturacion -> notificaciones, evitando depender de una unica transaccion distribuida.

Que monitorean Prometheus y Grafana?

Disponibilidad de endpoints, estado de contenedores, salud de servicios y metricas de infraestructura local.

Arquitectura de Sustentacion

Vista general

```
flowchart LR
    U[Cliente / Abogado] --> FE[Frontend Vercel]
    FE --> GW[NGINX API Gateway]
    GW --> AUTH[auth-service]
    GW --> CASES[cases-service]
    GW --> CLIENTS[clients-service]
    GW --> DOCS[documents-service]
    GW --> TIME[time-tracking-service]
    GW --> BILL[billing-service]
    GW --> NOTIF[notifications-service]
    GW --> DASH[dashboard-service]

    AUTH --> DB[(PostgreSQL / Supabase)]
    CASES --> DB
    CLIENTS --> DB
    DOCS --> DB
    TIME --> DB
    BILL --> DB
    NOTIF --> DB
    DASH --> AUTH
    DASH --> CASES
    DASH --> CLIENTS
    DASH --> DOCS
    DASH --> TIME
    DASH --> BILL
```

Flujo de denuncia

```
sequenceDiagram
    actor Cliente
    participant Frontend
    participant Gateway as API Gateway
    participant Cases as cases-service
    participant Docs as documents-service
    participant Billing as billing-service
    participant Notifications as notifications-service
    participant DB as PostgreSQL/Supabase

    Cliente->>Frontend: Completa formulario de denuncia
    Frontend->>Gateway: POST /api/cases
    Gateway->>Cases: Enruta solicitud
    Cases->>DB: Guarda caso en estado pending
    Cases-->>Frontend: Caso creado
    Cases->>Notifications: Evento caso_creado
    Notifications->>DB: Registra notificacion
    Cliente->>Frontend: Consulta Mis Casos
    Frontend->>Gateway: GET /api/cases
    Gateway->>Cases: Consulta casos
    Cases-->>Frontend: Lista de casos
```

SAGA funcional

```
stateDiagram-v2
    [*] --> DenunciaRegistrada
    DenunciaRegistrada --> RevisionAbogado
    RevisionAbogado --> DocumentosAsociados
    DocumentosAsociados --> TiempoRegistrado
    TiempoRegistrado --> FacturacionGenerada
    FacturacionGenerada --> NotificacionEnviada
    NotificacionEnviada --> [*]

    DocumentosAsociados --> RevisionAbogado: Error documento / reintento
    FacturacionGenerada --> TiempoRegistrado: Error facturacion / pendiente
```

NotificacionEnviada --> FacturacionGenerada: Error notificacion / reintento

Despliegue

```
flowchart TD
    Dev[Desarrollador] --> Git[GitHub main]
    Git --> Actions[GitHub Actions]
    Actions --> Tests[Instalacion, pruebas y build]
    Git --> Vercel[Vercel Frontend]
    Actions --> GHCR[Imágenes Docker GHCR]
    GHCR --> Docker[Docker Compose / Kubernetes]
    Docker --> Gateway[NGINX Gateway]
    Gateway --> Services[Microservicios]
    Services --> DB[(PostgreSQL / Supabase)]
```

Monitoreo con Prometheus y Grafana

La rubrica exige evidencia de monitoreo. Este proyecto incluye una base local para levantar Prometheus y Grafana junto con exportadores de infraestructura.

Levantar monitoreo

```
docker compose -f docker-compose.monitoring.yml up -d
```

Accesos

? Prometheus: <http://localhost:9090>

? Grafana: <http://localhost:3000>

? Usuario Grafana: admin

? Clave Grafana: admin

Metricas disponibles

? Disponibilidad de endpoints HTTP mediante blackbox-exporter.

? Recursos de contenedores mediante cAdvisor.

? Metricas de host mediante node-exporter.

? Metricas de PostgreSQL mediante postgres-exporter cuando se configure DATA_SOURCE_NAME.

? Estado de Prometheus y targets.

Evidencia para la sustentacion

1. Abrir Prometheus y mostrar Status > Targets.
2. Abrir Grafana y mostrar el dashboard LegalTech Overview.
3. Explicar que el monitoreo permite ver disponibilidad, consumo de recursos y salud de la plataforma.
4. Mostrar que los targets del frontend y gateway aparecen como UP si estan accesibles.